
CS 301

High-Performance Computing

Assignment 06 - Parallel Interpolation

Prem Kukadiya (202301452)
Suryadeepsinh Gohil (202301463)

April 19, 2026

Abstract

In this lab, we focus on parallelizing a scatter point-to-mesh interpolation algorithm using OpenMP. Mapping millions of randomly scattered points onto a structured grid in parallel introduces severe race conditions. Instead of using slow atomic locks, we implement a highly efficient "thread-local privatization" strategy. We analyze the performance across five different grid sizes and particle counts on both a Lab PC and an HPC Cluster, evaluating execution time, speedup, and parallel efficiency as we scale from 1 up to 16 cores (specifically 1, 2, 4, 6, 8, 10, 12, 14, and 16) to comprehensively track scaling behavior.

Contents

1	Implementation Approach and Race Conditions (Q1 & Q2)	3
1.1	The Race Condition Problem	3
1.2	Our Solution: Thread-Local Privatization	3
1.3	Pseudocode	3
1.4	Implementation Diagram	4
2	Hardware Specifications	4
3	Performance Analysis & Results	5
3.1	Execution Time and Speedup vs Cores (Q3)	5
3.2	Parallel Efficiency & Scalability (Q4)	6
3.3	Maximum Speedup Achieved (Q5)	6
4	Observations and Bottlenecks	6
4.1	Explaining the Speedup using HPC Knowledge (Q6)	6
4.2	Load Imbalance & Performance Bottlenecks (Q8)	6
5	Future Improvements and Alternative Approaches	7
5.1	Unlimited Resources Optimization (Q7)	7
5.2	Alternative Approaches & Data Structures (Q9 & Q10)	7
6	Conclusion	7

1 Implementation Approach and Race Conditions (Q1 & Q2)

1.1 The Race Condition Problem

When dealing with parallel interpolation, the scattered points are completely random. If we simply put a `#pragma omp parallel` for loop over the particles, multiple threads will likely process particles that are near each other. If two particles fall into the exact same grid cell, two threads will attempt to add their calculated weights to the exact same memory address (e.g., `mesh_value[base_idx]`) at the exact same time. This is a classic **race condition** that causes data corruption.

Using `#pragma omp atomic` before every single write operation would solve the corruption, but it would force threads to wait in line (serialize), destroying our parallel performance.

1.2 Our Solution: Thread-Local Privatization

To handle this without atomic locks, we used **Privatization**. Instead of writing directly to the main, global grid, every thread creates its own temporary, blank grid (local mesh). Each thread safely writes the weights of its assigned particles to its own private mesh. Once all particles are processed, the threads take turns (using a reduction phase) to add their local meshes together to form the final global mesh.

1.3 Pseudocode

Listing 1: Thread-Local Privatization Parallel Approach

```
1 Clear global_mesh entirely
2
3 #pragma omp parallel
4 {
5     // 1. Privatization: Allocate a blank grid for this specific thread
6     Create local_mesh of size GRID_X * GRID_Y initialized to 0
7
8     // 2. Parallel Work: Distribute particles across threads
9     #pragma omp for schedule(static)
10    for each particle p in all_particles {
11        Calculate grid index (i, j) for particle p
12        Calculate interpolation weights (w00, w10, w01, w11)
13
14        // Write to local memory (NO LOCKS NEEDED!)
15        local_mesh[idx] += w00
16        local_mesh[idx + 1] += w10
17        local_mesh[idx + GRID_X] += w01
18        local_mesh[idx + GRID_X + 1] += w11
19    }
20
21    // 3. Reduction Phase: Merge local meshes into the global mesh safely
22    #pragma omp critical
23    {
24        for each element in local_mesh {
25            global_mesh[element] += local_mesh[element]
```

```

26     }
27 }
28
29 Free local_mesh memory
30 }

```

1.4 Implementation Diagram

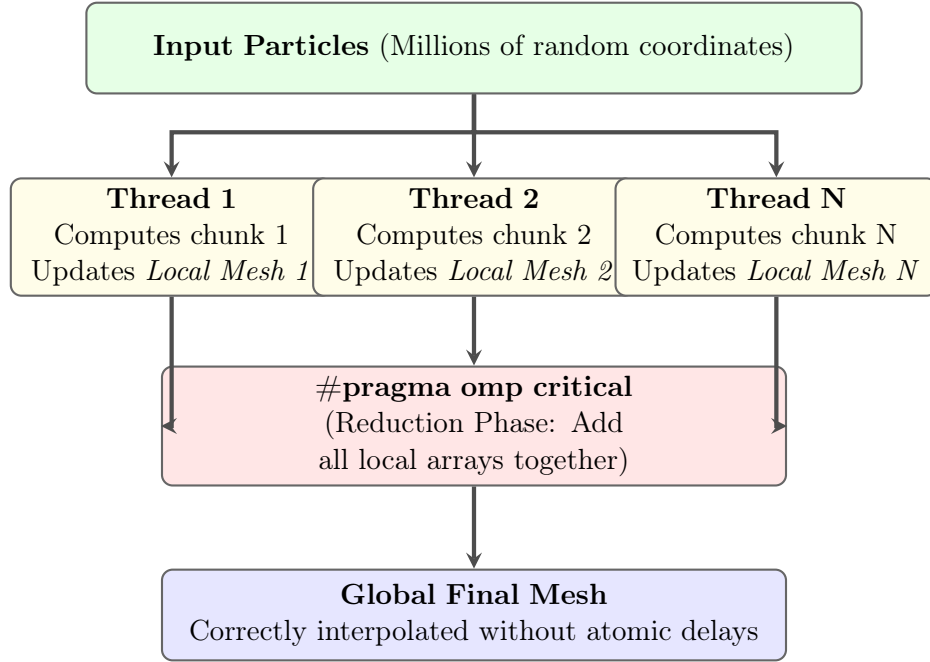


Figure 1: Visual diagram of our Thread-Local Privatization strategy. By giving each thread its own memory, we achieve zero write-contention during the heavy interpolation phase.

2 Hardware Specifications

The architecture of the systems was verified to understand baseline capabilities.

Table 1: Hardware Specifications Comparison

Parameter	Lab PC	HPC Cluster
CPU Model Name	12th Gen Intel(R) Core(TM) i5-12500	Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40GHz
Total CPUs (Threads)	12	24 (Dual-socket)
L1d Cache	288 KiB	32 KiB
L2 Cache	7.5 MiB	256 KiB
L3 Cache	18 MiB	15 MiB

3 Performance Analysis & Results

3.1 Execution Time and Speedup vs Cores (Q3)

The following plots illustrate the performance scaling for all five configurations. The graphs present a side-by-side comparison between the Lab PC and the HPC Cluster, tracking performance across 1, 2, 4, 6, 8, 10, 12, 14, and 16 cores.

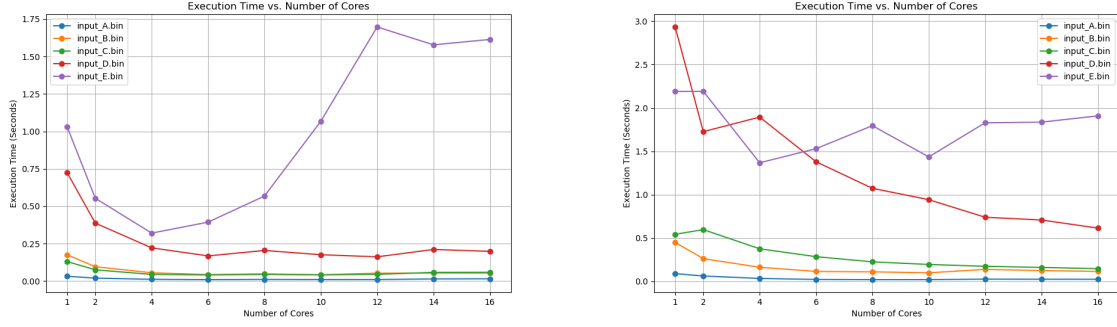


Figure 2: Execution Time vs Cores on Lab PC (left) and HPC Cluster (right).

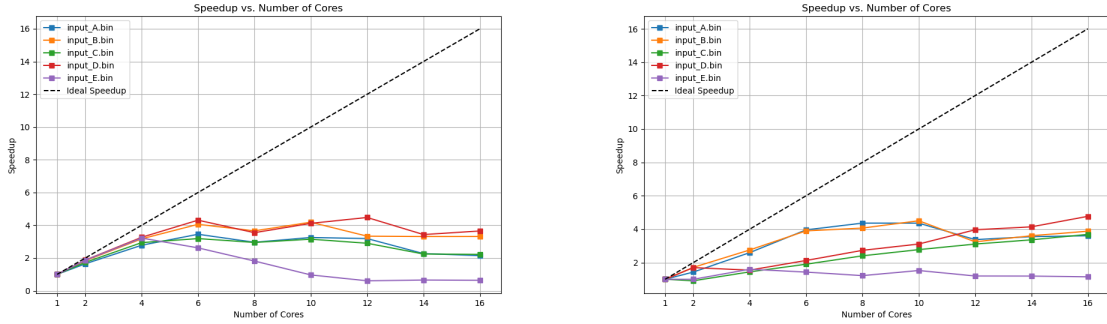


Figure 3: Speedup vs Cores on Lab PC (left) and HPC Cluster (right).

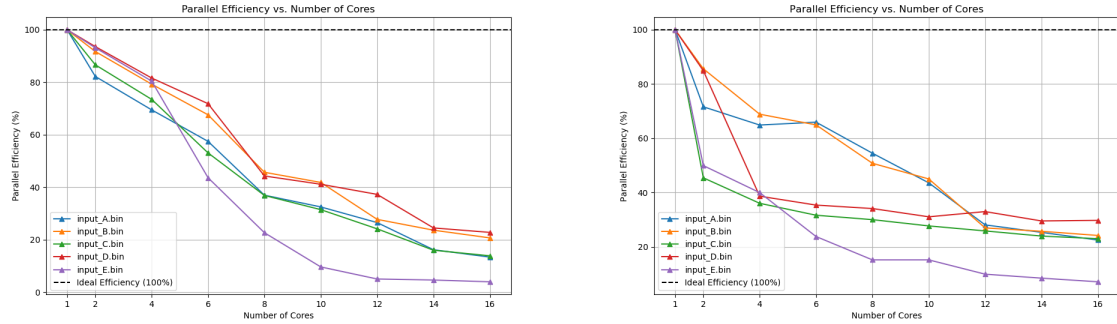


Figure 4: Parallel Efficiency vs Cores on Lab PC (left) and HPC Cluster (right).

3.2 Parallel Efficiency & Scalability (Q4)

Parallel efficiency tells us how well the hardware is being utilized as we add more cores (Efficiency = $\frac{\text{Speedup}}{\text{Number of Cores}} \times 100$).

- **Good Scalability (Configs A, B, D):** For normal grid sizes, the efficiency starts high (85-90% on 2 cores) and steadily decreases as synchronization overheads rise. However, the execution time keeps dropping steadily as we scale toward 16 cores, showing decent scalability.
- **Poor Scalability (Config E):** For Config E (1000 × 400 grid, 14 million points), scalability completely collapses. On the Lab PC, 16 cores yield a speedup of **0.64x** (slower than serial!). Efficiency drops to just 4.0%. This severe degradation is caused by cache limitations, analyzed further in Section 4.

3.3 Maximum Speedup Achieved (Q5)

By comparing the parallel times against our strict serial baseline, we recorded the following maximum speedups:

- **HPC Cluster:** Maximum speedup of **4.77x** achieved on Config D using 16 Cores.
- **Lab PC:** Maximum speedup of **4.48x** achieved on Config D.

The HPC cluster scales slightly better at higher thread counts because the Xeon architecture features superior memory channel bandwidth specifically designed for multi-threading compared to a consumer desktop i5 processor.

4 Observations and Bottlenecks

4.1 Explaining the Speedup using HPC Knowledge (Q6)

Why did we max out around $\sim 4.77x$ speedup even when using 16 cores? Interpolation is heavily **Memory-Bound**. The CPU can calculate the math (multiplication for weights) much faster than it can fetch and write data from the RAM to the cache. As we add more threads, they all compete for the same memory bus bandwidth. Once the memory bus is saturated (around 4-6 cores), adding more cores provides severely diminishing returns. The CPU cores sit idle waiting for data to arrive from RAM.

4.2 Load Imbalance & Performance Bottlenecks (Q8)

Regarding **Load Imbalance**, we utilized `#pragma omp for schedule(static)` to distribute the particle processing. Because the scattered points are uniformly random across the spatial domain, static scheduling inherently provides a well-balanced computational load among threads. If the particles were heavily clustered in one corner, spatial domain decomposition would suffer from severe load imbalance, but our particle-based privatization method naturally avoids this spatial penalty.

However, the most glaring performance bottleneck is the massive drop in **Config E** (1000 × 400 grid). Our approach uses *Privatization*, meaning every thread gets its own copy of the mesh. A

1000 × 400 mesh contains 400,000 double-precision floats (~ 3.2 MB). If we use 16 threads, we are suddenly allocating $16 \times 3.2 = \mathbf{51.2\ MB}$ of memory inside the parallel block!

This massively exceeds the L3 Cache size of both the Lab PC (18 MB) and the Cluster (15 MB). This causes severe **Cache Thrashing**. The CPU is forced to constantly swap data to the slow main RAM, which destroys performance and causes the parallel code to run slower than the serial code (as seen in the Lab PC's 0.64x speedup).

5 Future Improvements and Alternative Approaches

5.1 Unlimited Resources Optimization (Q7)

If given unlimited HPC resources (supercomputers with distributed nodes and GPUs):

- **GPU Offloading (CUDA/OpenACC):** GPUs have thousands of cores and massive memory bandwidth (HBM2). Interpolation is highly parallelizable on GPUs using fast atomic additions in shared memory.
- **MPI + OpenMP (Hybrid):** I would divide the spatial domain across multiple computer nodes using MPI (Message Passing Interface). Each node would handle a smaller sub-grid using OpenMP, completely eliminating the L3 cache exhaustion problem we saw in Config E.

5.2 Alternative Approaches & Data Structures (Q9 & Q10)

To fix the memory bandwidth limits and cache thrashing without changing the hardware, we could change our data structures:

- **Domain Decomposition (Spatial Partitioning):** Instead of every thread having a full copy of the mesh (which ruins cache), we can divide the physical mesh into chunks. Thread 1 exclusively owns the top-left quadrant, Thread 2 owns the top-right, etc. This requires zero privatization and no reduction phase, drastically improving memory efficiency.
- **Particle Sorting / Binning (Bonus Data Structure):** We could first sort the scattered `Points` array into bins based on which grid cell they belong to. By creating an array of "Linked Lists" or "Buckets" representing grid cells, a thread can process an entire cell's worth of particles at once. This maximizes **Cache Locality** (since the thread stays focused on one localized piece of memory) and prevents false sharing between cache lines.

6 Conclusion

In this assignment, we learned how to use multiple computer cores to make a program run much faster. We used OpenMP to divide the work. To stop the cores from writing over each other's work at the same time, we gave each core its own temporary grid. This worked very well for smaller grids, making the program almost 5 times faster.

However, we found a big problem when the grid got too large. Giving every core a full copy of a huge grid used up all the computer's fast memory (called the cache). This caused the computer to slow down a lot, making the parallel code even slower than the normal serial code.

Overall, we learned that writing good parallel code is not just about adding more cores to a problem. You also have to be very careful with how the computer stores and reads memory. To fix this in the future, we would split the grid into smaller puzzle pieces so the memory never gets overloaded.